

DTS · Documento técnico

Versión documental: 2026-09-26. Entorno: QA. DTS significa Desarrollo & Tecnología Santander.

Arquitectura y componentes

Una aplicación Flutter comparte interfaz y lógica para web, Android e iOS. Firebase Authentication identifica al usuario mediante Google o correo y contraseña. Cloud Functions valida el correo verificado, el perfil activo, el rol y la membresía en cada operación. La aplicación utiliza la función callable `api`; no escribe directamente en Firestore. Firebase Storage almacena evidencias con reglas propias. Hosting publica la web; Firebase Cloud Messaging (FCM) entrega push.

Versiones reproducibles: Flutter 3.41.6, Dart 3.11.4, Functions Node 22, TypeScript, Firebase Admin y Firebase Functions. Consultar los archivos `lock` para las versiones exactas de dependencias. Firestore reside en `nam5`; Functions, en `us-central1`. El proyecto QA es `sistema-de-gestion-y-pqrs`.

Componente	Responsabilidad
<code>lib/core/api.dart</code>	Transporte callable, sesión y contexto empresa/proyecto; descarta respuestas obsoletas al cambiar rápidamente de empresa.
<code>lib/core/notifications.dart</code>	Registro de instalación, destino web/móvil, token FCM, permisos, recepción y cierre de sesión.
<code>lib/features/auth.dart</code>	Acceso, registro, verificación, recuperación, vinculación Google e invitaciones.
<code>lib/features/shell.dart</code>	Navegación según rol, contexto y aviso recibido en primer plano.
<code>resources.dart</code>	Formularios y consultas de recursos operativos, cobertura y renovaciones.
<code>tickets.dart</code>	Radicación, filtros, detalle, comentarios, evidencias y transiciones.
<code>admin.dart</code> , <code>notification_widgets.dart</code>	Accesos, cuenta, dispositivos, bandeja y trazabilidad de envíos.
<code>imports.dart</code>	Inspección y revisión de catálogos importados.
<code>functions/src/index.ts</code>	Autorización, operaciones de negocio, transacciones, auditoría y funciones desplegadas.
<code>functions/src/domain.ts</code>	Esquemas Zod, permisos, estados y cálculo de tiempo hábil.
<code>functions/src/notifications.ts</code>	Destinos privados, eventos pendientes, destinatarios, envío, reintentos y recibos.
<code>firestore.rules</code> , <code>storage.rules</code>	Restricciones para accesos directos, archivos y separación de audiencias.

Módulos y entidades

Módulo	Entidades y funciones
Empresas	companies: nombre obligatorio; datos legales, NIT, sector, dirección, contacto, horario, zona, estado y etiquetas opcionales.
Proyectos	projects: empresa, producto, objetivo, alcance, estado, versión, ambiente, responsables y fechas.
Productos e instalaciones	products, installations: soluciones reutilizables, instalación, versión, ambiente, URL y fecha de despliegue.
Personas y estructura	contacts: cargo, área, dependencia, sede, disponibilidad, responsabilidad, canal y contacto principal.
Catálogo y conocimiento	catalog, articles: descripción funcional, módulo, preguntas frecuentes, roles, versión y publicación.
Contratos	contracts, publicContracts: vigencia, valores, firmantes, condiciones, cobertura y SLA; proyección pública separada.
Servicios	services, publicServices: proveedor, titularidad, costos/precios, vigencia, renovación, referencias y proyección pública. Nunca claves de infraestructura.
Implementación y agenda	events: tarea, hito, capacitación, visita, reunión, despliegue y compromiso, con fecha, responsable, asistentes y material.
Valores	finance: ingresos, gastos y participaciones; no es un sistema contable.
Solicitudes/PQRS	tickets: número consecutivo transaccional, categoría, impacto, prioridad, responsable, visibilidad, cobertura, SLA y tiempos.
Acceso	users, memberships, invitations: perfil, rol, permisos, empresa/proyecto y alcance de solicitudes.
Enlace de proyectos	repositories, imports, catalogReviews: propuesta funcional a partir de archivos o repositorio; publicación tras revisión.
Notificaciones	notificationEvents, notifications, subcolección deliveries: evento de negocio, bandeja personal y evidencia de envío por canal.
Dispositivos	`userDevices/{uid}/slots/{web
Auditoría	audit: actor, acción, entidad, fecha, resumen y contexto de dispositivo cuando existe.

Roles y aislamiento

El propietario tiene acceso completo, incluido material técnico y gestión de identidades. El comercial accede a la operación de todas las empresas, pero no a notas técnicas ni repositorios. El técnico requiere membresía y permiso `technical` para material técnico. Cliente y lector solo consultan empresas/proyectos asignados y contenido publicado; el lector no comenta ni cambia estados. El permiso adicional `manage` habilita operaciones de gestión y debe otorgarse con criterio administrativo.

La membresía admite todos los proyectos o una lista de hasta 30. El alcance requester restringe las solicitudes del cliente a aquellas radicadas por su cuenta. La visibilidad requester del ticket también lo restringe. El servidor vuelve a consultar el perfil y membresías; no confía exclusivamente en claims almacenados en el dispositivo.

Notas y evidencias se separan en `public`, `internal` y `technical`. Una notificación técnica jamás se convierte en un aviso público. El propietario inicial `jairoandresorostegui@gmail.com` fue provisionado explícitamente. El registro público o el dominio del correo nunca otorgan administración.

API callable

Formato: `{action, data, clientContext?: {sessionId}}`. Todas las acciones, salvo aceptación de invitación que tiene su propia validación, requieren perfil activo y correo verificado. `clientContext` sirve para trazabilidad; no concede permisos.

Grupo	Acciones
Sesión y datos	<code>session</code> , <code>list</code> , <code>save</code> , <code>dashboard</code>
Solicitudes	<code>createTicket</code> , <code>ticketDetail</code> , <code>comment</code> , <code>transition</code> , <code>ticketNotifications</code>
Administración	<code>access</code> , <code>invite</code> , <code>acceptInvitation</code>
Importación	<code>previewImport</code> , <code>githubImport</code> , <code>approveImport</code>
Vigencias	<code>renew</code>
Dispositivos	<code>registerDevice</code> , <code>updateDeviceToken</code> , <code>unregisterDevice</code> , <code>myDevices</code>
Bandeja y recibos	<code>notificationDetail</code> , <code>notificationReceipt</code> , <code>markRead</code> ; listado con <code>list</code> y colección <code>notifications</code>

Los esquemas Zod son la definición ejecutable de campos, límites y enumeraciones. Listados paginados evitan offsets. `save` admite `updatedAt` para detectar edición concurrente. Errores de validación, permisos, inexistencia y transición se devuelven con códigos callable y mensajes en español. No se registran cuerpos de solicitud ni tokens en los logs de fallos.

Ciclo de solicitudes y SLA

Estados: nuevo, clasificación, esperando cliente, primer nivel, escalado, análisis, ejecución, resuelto, cerrado, reabierto y cancelado. La matriz exacta está en `domain.ts`. Una ruta habitual es nuevo → clasificación → primer nivel → escalado → análisis → ejecución → resuelto → cerrado. Las opciones permitidas dependen del estado y rol; el cliente solicitante confirma el cierre o reabre dentro del plazo permitido de 30 días desde resolución.

El contrato activo de soporte determina la cobertura y los objetivos en minutos hábiles. El cálculo considera horario, días, festivos y zona del contrato. Esperando cliente pausa el objetivo según tiempo hábil; reanudar ajusta las fechas. Sin cobertura se conserva la solicitud y se etiqueta fuera de cobertura. Los comentarios admiten hasta cinco adjuntos de hasta 10 MB cada uno, con MIME y rutas autorizadas.

Último celular y última web

Cada acceso autorizado registra una instalación aleatoria local y genera una sesión en el servidor. Web significa navegador, incluso si se usa desde un teléfono; celular significa la aplicación Android/iOS. No se recogen IMEI, identificadores publicitarios ni ubicación. El nombre de equipo/plataforma procede del cliente y es orientativo, no una prueba forense de hardware.

Un nuevo acceso reemplaza únicamente su tipo. Actualización de token y cierre de sesión comparan la sesión con el destino actual en una transacción. Un destino antiguo no puede recuperar prioridad mediante un refresco tardío ni borrar el destino nuevo. Al cambiar de cuenta en el mismo navegador, el índice hash del token retira el destino de la cuenta anterior. Los tokens solo están en colecciones privadas; myDevices entrega un resumen sin token ni identificador de instalación.

El registro de acceso funciona aunque no se conceda push. El usuario activa permisos en Mi cuenta. Al cerrar sesión se desactiva el destino correspondiente; si falla la conexión se intenta invalidar el token local y se cierra la sesión de todas formas. El servidor registra los orígenes de operaciones con sesión válida. Accesos realizados antes de esta versión o clientes sin contexto aparecen sin dispositivo registrado.

Notificaciones y concurrencia

Crear una solicitud, comentar (incluyendo evidencia/tiempo) o cambiar de estado crea el evento de notificación dentro de la misma transacción que el cambio de negocio y auditoría. Un fallo de esa transacción no deja un aviso de un cambio inexistente. Incluye al actor cuando pertenece a la audiencia; así conserva su propia trazabilidad.

notificationDispatch procesa la creación de notificationEvents. Cada evento produce una bandeja individual para propietario/comerciales y miembros autorizados del proyecto, respetando audiencia y visibilidad. Los avisos de vencimiento se adaptan mediante notificationLegacy. notificationRetry revisa pendientes cada cinco minutos. El recordatorio de vigencias se ejecuta diariamente a las 07:00 de Bogotá con umbrales de 30, 7, 1 y 0 días y claves deterministas.

Las bandejas usan un ID determinista evento/destinatario. Cada aviso tiene dos resultados de envío, web y mobile. Transacciones y bloqueos temporales evitan trabajo simultáneo sobre el mismo envío. Los errores transitorios se reintentan con espera exponencial, hasta seis intentos; un token inválido se elimina. Los permisos se reevalúan al preparar destinatarios, enviar, listar y abrir. Renovar invalida un recordatorio antiguo pendiente por cambio de fecha.

Estado	Significado
Registrado / pendiente	El evento y/o la bandeja existen. Puede estar preparando destinos.
Enviando / reintento	Se procesa el destino o se espera el siguiente intento.
Aceptado por Firebase	FCM aceptó la petición. No demuestra recepción física.
Recibido	La aplicación en primer plano confirmó el mensaje.

Estado	Significado
Abierto	El destinatario abrió el aviso y la aplicación confirmó el origen.
Leído	El usuario abrió el detalle o marcó el aviso como leído.
Sin permiso / sin destino	Existe bandeja, pero no un destino push utilizable.
Destino vencido / fallido	Error persistente visible en la trazabilidad.
Cancelado	Permiso revocado o recordatorio que perdió vigencia.
Histórico sin reenvío	Migración de avisos anteriores; no genera push retroactivo.

FCM y los eventos de infraestructura ofrecen entrega con posibles repeticiones. Si un proceso cae después del envío y antes de registrar la aceptación, un reintento puede repetir el push. Se usan tags/collapse IDs estables para reducir duplicados visuales. No se promete entrega exactamente una vez. Los recibos son idempotentes y preservan su primera fecha. Un aviso no confirmado se conserva; no se inventan fechas de lectura.

La pantalla bloqueada recibe un texto genérico sin asunto ni contenido privado. El detalle se abre tras autenticación y autorización. La web usa `firebase-messaging-sw.js` con Firebase JS 12.19.0. Flutter permite una clave pública VAPID mediante `--dart-define=FCM_VAPID_KEY=...`; si se omite, utiliza el comportamiento predeterminado del SDK. Un cambio del SDK requiere revisar la versión del service worker. Los dispositivos sin push conservan la bandeja.

Configuración, pruebas y despliegue

```
flutter pub get
npm --prefix functions ci
flutter analyze
flutter test
npm --prefix functions run build
$env:FUNCTIONS_EMULATOR='true'
$env:GCP_PROJECT='demo-dts-gestion'
$env:FUNCTIONS_DISCOVERY_TIMEOUT='60'
firebase emulators:exec --only 'auth,firestore,functions,storage' --project demo-dts-gestion 'npm --prefix functions test'
flutter build web --release --no-wasm-dry-run
node scripts/check-web-plugins.cjs
firebase deploy --only 'functions,firestore,storage,hosting' --project sistema-de-gestion-y-pqrs
flutter build apk --debug
```

El primer despliegue de funciones con política de reintentos requiere `--force` en ejecución no interactiva; los procesadores están preparados para repeticiones. Confirmar siempre el ID explícito de QA. Los índices se regeneran con `node scripts/generate-indexes.cjs`. GitHub Actions valida Flutter, Functions, reglas/emuladores y compilación iOS sin firma en macOS. Las pruebas reales adicionales son `scripts/qa-smoke.cjs`, `scripts/qa-navigation.cjs` y `scripts/qa-notifications.cjs`. Sus identidades temporales se eliminan; la auditoría conserva evidencia de ejecución sin credenciales.

Después de agregar plugins, verificar el registro generado con `check-web-plugins.cjs`. Si detecta una caché obsoleta, regenerar la caché Flutter de compilación y compilar nuevamente antes de desplegar. No editar manualmente los registrantes generados. El inicio de sesión guarda el destino; la conexión FCM continúa de forma asíncrona y limita la espera por token.

Android usa el canal `dts_updates`, permiso de notificaciones y paquete `co.com.dts.dts_gestion`. El APK debug sirve para QA. iOS usa `co.com.dts.dtsGestion`, entitlement push y modo remoto en segundo plano; requiere equipo Apple, firma y clave APNs en Firebase para enviar a un dispositivo real. Web requiere HTTPS y compatibilidad del navegador.

Referencia oficial: [Configuración FCM Flutter](#), [Recepción de mensajes](#), [Gestión de tokens](#).

Operación y límites

Revisar `notificationEvents` pendientes antiguos y `deliveries` fallidos, logs de Functions y ejecuciones Scheduler. La consola privada puede inspeccionar códigos de error; no copiar tokens a reportes. La aplicación muestra un historial por solicitud con destinatarios para administradores y con la propia cuenta para los demás. Auditoría completa se consulta mediante API autorizada; no hay todavía un visor especializado de auditoría en el menú.

No hay proveedor de correo externo configurado para avisos de negocio. No hay distribución firmada en App Store/Play Store. Repositorios privados requieren GitHub App/Secret Manager; la importación local y revisión manual sí están disponibles. No hay webhook de sincronización automática de código. La política comercial de retención, borrado/archivo, copias de seguridad y recuperación debe definirse antes de producción; esta versión conserva historial y sesiones sin TTL automático. La prueba QA no certifica rendimiento a gran escala ni todas las combinaciones posibles de hardware, conectividad y sistema operativo.